

Pedestrian Navigation Without GPS

Project Number: 25-1-1-3330

Project Report

Student: Rabeea Bulbul ID: 318979432

Student: Siba Aasi ID: 213780638

Supervisor: Dr. Gabriel Davidov

Project Carried Out at: Tel Aviv University

Contents

Abstract **3**

1. Introduction **4**

1.1 Goals 4

1.2 Motivation 4

1.3 Our approach and setup 4

2. Theoretical background **6**

2.1 Accelerometer 6

2.2 Gyroscope 6

2.3 Barometer 7

2.4 Magnetometer 7

2.5 Optical flow 7

2.6 Filtering and step detection 8

2.7 Connecting the phone to the laptop 8

2.8 Pedestrian dead reckoning 9

3. Simulation **10**

3.1 The offline workflow 10

3.2 The experiments we recorded 10

3.3 Detecting steps 10

3.4 Detecting crab steps 12

3.5 Detecting stairs and the elevator 12

3.6 Detecting turns 13

3.7 Reconstructing the 3D path 14

4. Implementation **15**

4.1 The real-time system 15

4.2 Detecting steps and turns 16

4.3 Deciding direction with the camera 16

4.4 Stairs and the elevator 17

4.5 Step-length calibration 17

4.6 Voice feedback 17

4.7 Outputs 17

4.8 Main parameters 17

5. Results and tests **19**

5.1 Step count and distance 19

5.2 Movement classification 19

5.3 Stairs and the elevator 20

5.4 Status against the objectives 21

6. Summary, conclusions and further work **22**

6.1 Summary 22

6.2 What we learned 22

6.3 Further work 22

7. Project documentation **23**

References **23**

Abstract

This project tracks a walking person indoors, in real time, without GPS. We mount an Android phone on the chest and stream its accelerometer, gyroscope, barometer and camera to a laptop. The laptop detects each step and classifies the movement (walking forward or backward, sideways "crab" steps, 90° turns, stairs and elevator) and draws the walked path on a live 3D map.

Steps are counted from the accelerometer. The camera decides forward from backward, because the image expands when we move forward and contracts when we move backward. The barometer detects stairs and elevators. We calibrate the step length once per user and announce each movement by voice. On our test runs the system reached 1.2% step-count error, 96.9% movement classification on mixed routes, path error of 0.24m per 10m, and correctly counted an 18-stair flight (+3.06 m) and a four-floor elevator ride (+11.29 m).

1. Introduction

1.1 Goals

Our goal was to build a system that tracks a pedestrian in real time without GPS, using only a smartphone's sensors and camera. It should recognise the type of each movement, update a 3D map while the person is still walking, and work for different users after a short calibration. The quantitative targets we set at the start were:

- Step-count error of 5% or less.
- Path error of at most 1 m for every 10 m walked.
- Movement-classification accuracy of at least 85%.
- Reliable performance for at least 4 different users.

The movements we wanted to recognise are forward and backward steps, crab steps to the left and right, 90° turns, climbing and descending stairs, and riding an elevator up or down.

1.2 Motivation

Almost every navigation system today depends on GPS, but GPS is not always available or reliable. Indoors, in underground floors, in tunnels and in stairwells the satellite signal is weak or missing, and it can also be jammed or spoofed, which has become a common experience in our region in recent years. In all of these situations a person still needs to know where they are.

We wanted a system that keeps working exactly there, using only the phone a person already carries, with no extra hardware and no network. Such a system is useful for rescue teams moving inside buildings and tunnels, and for navigating inside a large building such as a hospital or a mall. Because our system also announces each movement out loud, we see it as a first step toward a guidance aid for visually impaired users, who need exactly this kind of turn-by-turn indoor feedback.

1.3 Our approach and setup

We split the work into two semesters. In the first semester we worked completely offline: we recorded controlled walks, saved them as files, and studied them in MATLAB until we understood how every movement looks in the raw signals and which filter and thresholds separate a real step from noise. In the second semester we moved everything to real time, rewrote the processing in Python, added the camera to decide walking direction, and built the live 3D map and the voice feedback.

We fixed the phone to the chest, standing upright (portrait), with the screen facing forward, and kept this exact mounting for every experiment. We chose the chest for three reasons: it leaves the hands free, it holds the phone steady so the body's sway is not amplified the way it is in a hand or a loose pocket, and it points the camera straight ahead at the scene we are walking into. Keeping the phone upright and unrotated also means each sensor axis always has the same physical meaning: the vertical axis feels the step bounce, the sideways axis feels crab steps, and the gyroscope's vertical axis feels turns. This is what makes the detection reliable.



Figure 1. Our setup: the phone on the chest, upright, screen facing forward. We keep this exact mounting in every experiment.

The experiments in this book were carried out in a building at Tel Aviv University, using its corridors, a staircase and an elevator. The main assumptions behind the system are:

- The phone is on the chest, upright, screen forward, and is not rotated while walking.
- The user walks at a normal, even pace, and turns are about 90°.
- One direction per walking segment (we do not mix forward and backward without a turn or a pause between them).
- A stair flight is climbed without stopping in the middle, and during an elevator ride the user stands still.

2. Theoretical background

This chapter explains the sensors we used, the processing we apply to their signals, and the two ideas at the heart of the system: optical flow and dead reckoning. For each sensor, we also explain how it is used in our system.

2.1 Accelerometer

The accelerometer is a small MEMS sensor that measures acceleration along three perpendicular axes. Phones also provide a "linear acceleration" signal with gravity already removed, which is the one we use. Inside the sensor a tiny proof mass is held by springs, and when the phone accelerates the mass shifts and the sensor reads that shift as acceleration. Because our phone is fixed upright on the chest, the axes always mean the same thing: the Y axis is vertical, X is sideways and Z points out of the screen.

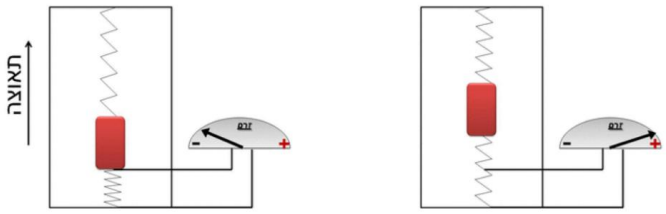


Figure 2. An accelerometer measures acceleration from the movement of a small proof mass held by springs.

Every step is a small vertical bounce of the body, so it appears as one clean peak on the Y axis, and this is how we count steps. Crab (sideways) steps push the body sideways, so we read them from the X axis instead. We chose the X axis for crab steps because their sideways peaks there are clearly taller than the small side-to-side sway of ordinary walking, which lets us tell the two apart.

2.2 Gyroscope

The gyroscope measures how fast the phone is rotating, in rad/s. When the walker turns, the phone rotates around the vertical (yaw) axis, and this appears as a clear pulse whose sign tells us left from right. Since we assume 90° turns, we do not need to integrate the rotation into an exact angle. Detecting the pulse and its direction is enough.

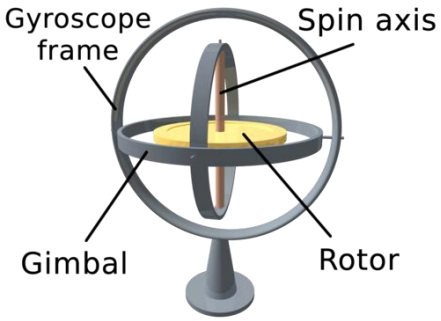


Figure 3. A gyroscope measures rotation rate. In our system the useful rotation is yaw, the turning of the body left or right about the vertical axis.

2.3 Barometer

The barometer measures air pressure, which drops as we go higher. We turn pressure into a height relative to the start of the run using the standard barometric formula:

$$h = 44330 * (1 - \frac{p}{p_0})^{0.1903}$$

where p_0 is the pressure measured at the start. Because stairs and elevators are height changes, this is the sensor we use to detect them. The catch, as we explain later, is that the noise of the barometer is about the same size as a single stair step, and this is why we detect whole flights instead of single stairs.

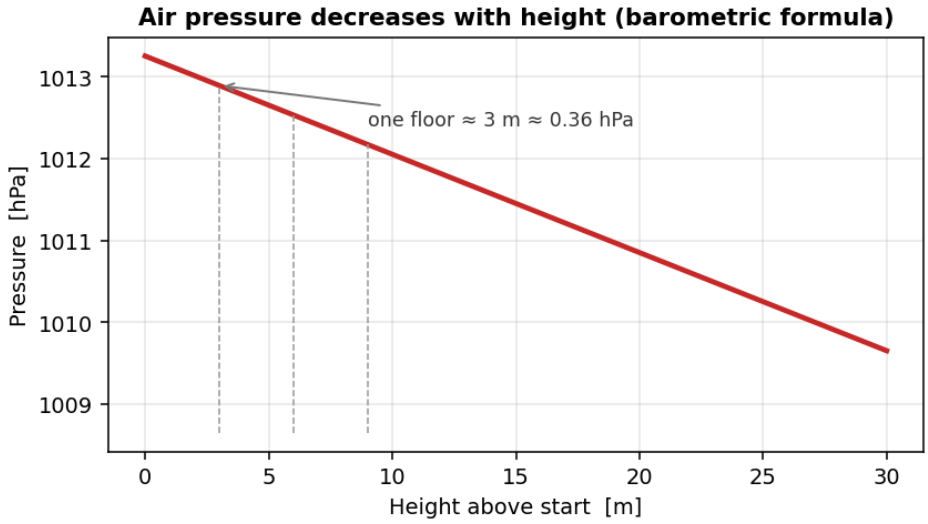


Figure 4. Air pressure falls with height. One floor (about 3 m) changes the pressure by only about 0.36 hPa, which is close to the sensor's own noise.

2.4 Magnetometer

The magnetometer measures the direction of the magnetic field and can act as a compass, which would give an absolute heading and allow turns of any angle. We recorded it during the offline stage and considered using it, but in the end we did not. Indoors the magnetic field is bent by the building's metal structure and its electrical equipment, so the heading it reports is unreliable, and once we fixed our turns at 90° the gyroscope alone was enough. We mention it because it is the natural place to start if the system is later extended to arbitrary turn angles.

2.5 Optical flow

Optical flow is the pattern of apparent motion of the image between two camera frames. The idea is that as the camera moves, each point in the picture appears to move, and these small movements form a flow field of little arrows across the image.

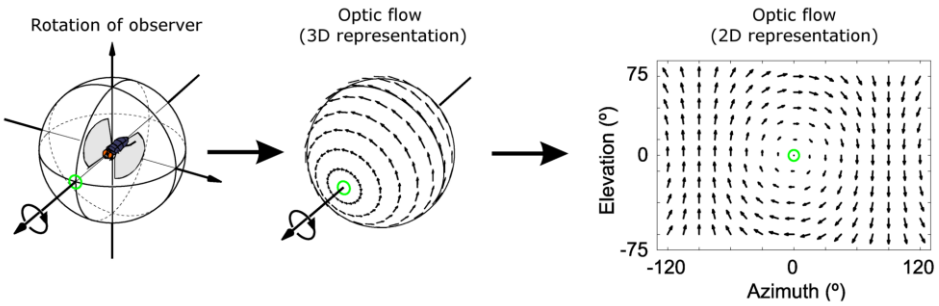


Figure 5. Optical flow: as the observer moves, points in the image appear to move, forming a flow field (shown here in 3D and in a flat 2D view).

We use optical flow to answer the one question the accelerometer cannot: is a step forward or backward? When the camera moves forward the whole image expands away from the centre, and when it moves backward the image contracts. We measure this expansion (the radial divergence of the image) with the Farneback optical-flow algorithm. A positive value means forward, and a negative value means backward.

2.6 Filtering and step detection

Raw acceleration is noisy, so before we look for steps we band-pass filter it to the walking band (0.45 to 3 Hz) with a Butterworth filter, applied forward and then backward so it adds no time shift. This leaves one clean oscillation per step. The gyroscope gets a gentle Butterworth low-pass to keep the turn pulse clean, and the barometer is smoothed with a short moving-average, since for height we only care about the slow changes and not the fast detail.

We then take the steps as the peaks of the filtered signal that pass three tests: a minimum height, a minimum prominence (how far the peak stands out from its neighbours), and a minimum time gap between peaks. We set these thresholds by trial on our own recorded walks.

2.7 Connecting the phone to the laptop

On the phone we run the IP Webcam application, which turns the phone into a small web server: it publishes the current sensor readings at the address /sensors.json and a camera snapshot at /shot.jpg. The laptop simply asks for these two addresses in a loop.

We first connected over Wi-Fi, with the phone and laptop on the same network and the laptop reading the phone by its IP address (for example http://10.0.0.2:8080). This worked, but Wi-Fi turned out to be the weakest part of the system. The university network dropped often, and inside the elevator, which is one of the movements we most wanted to measure, there is no Wi-Fi at all, so the connection died exactly when we needed it. We solved this by connecting the phone to the laptop with the ordinary charging cable and using USB tethering, which creates a small private network between the two devices over the cable. The same requests then work everywhere, including inside the elevator. The system still supports Wi-Fi as an option, but we prefer the cable for its reliability.

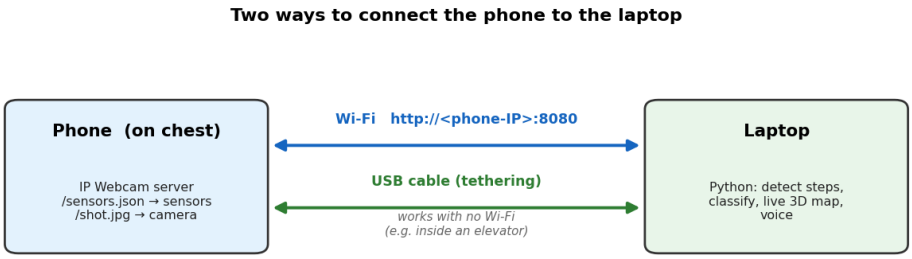


Figure 6. We can connect over Wi-Fi (by the phone's IP address) or over the USB cable. The cable keeps working where Wi-Fi does not, such as inside an elevator.

2.8 Pedestrian dead reckoning

Dead reckoning builds the path by adding one step at a time. For every detected step we move the position forward by one step length in the current heading:

$$x_{k+1} = x_k + L \cdot d_k$$

where L is the step length and d_k is the heading, which rotates by 90° on each turn. Crab steps move the position sideways, and stairs or the elevator change the height. The weak point of dead reckoning is that errors add up, and there is no GPS to reset them, so a correct classification and a correct step length matter a great deal. This is why we spent so much of the project on exactly those two things.

3. Simulation

Everything started offline. Before we connected anything in real time, we recorded walks, took them back to the computer, and studied them in MATLAB until the detection made sense. All the figures in this chapter come from that offline analysis.

3.1 The offline workflow

For the offline work we recorded the phone's sensors with the Sensor Logger application. We would perform one experiment (for example, walking ten steps forward) while Sensor Logger recorded the accelerometer, gyroscope and barometer, then export the recording as CSV files, one table per sensor with a timestamp on every sample, and copy them to the computer. We then opened the files in MATLAB, with no phone connected, and processed them there. Working from a saved file instead of a live stream was the key to this stage: we could replay the exact same walk as many times as we wanted and try different filters and thresholds on identical data until the result matched what we had really done.

3.2 The experiments we recorded

We recorded a set of short, controlled walks, each one aimed at a single movement so we could see its signature clearly: forward walking of a known number of steps, backward walking, crab steps to the left and right, climbing and descending a known number of stairs, an elevator ride of several floors, and a few mixed routes that combined everything. For each run we wrote down what we actually did, so we could compare it against what the analysis found.

3.3 Detecting steps

We start from the raw accelerometer. It carries all three axes at once, mixed with noise and body vibration, so the steps are there but hard to count directly.

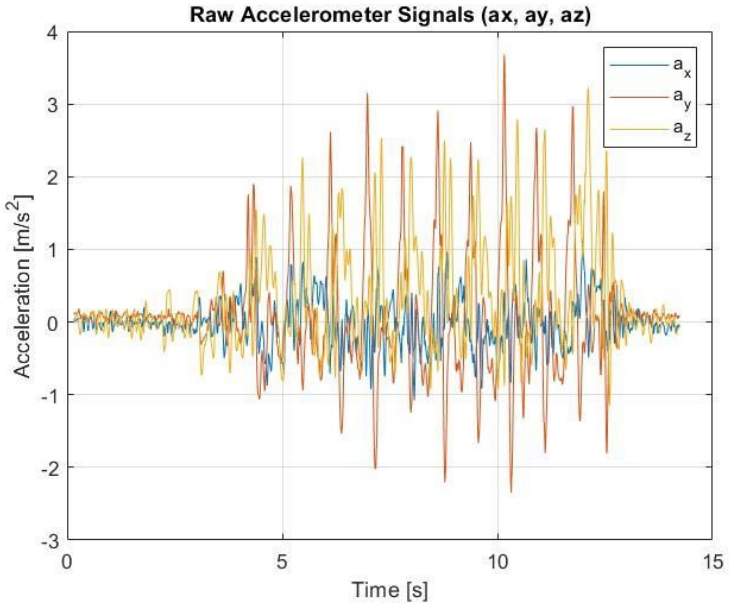


Figure 7. Raw accelerometer (all three axes) during a forward walk. The step pattern is buried in noise and spread across the axes.

We take the vertical (Y) axis and band-pass filter it. This removes the noise and the slow drift and leaves one clean oscillation per step.

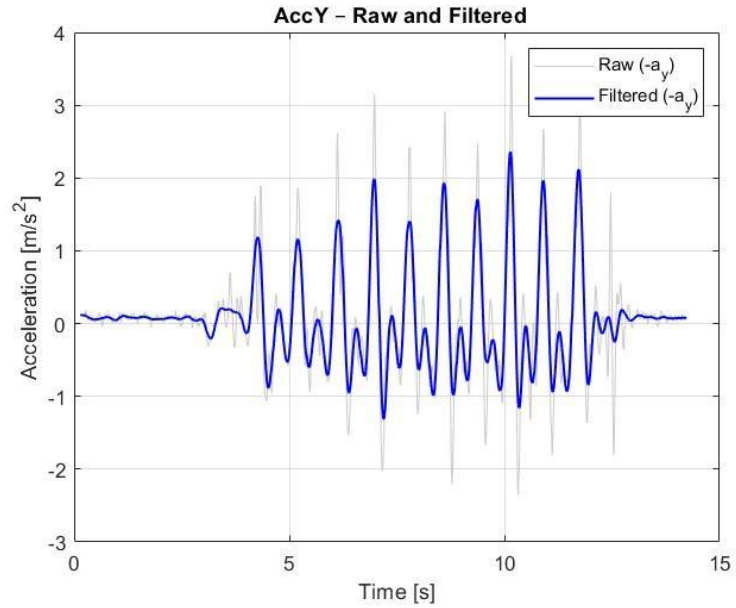


Figure 8. The vertical axis before (grey) and after (blue) filtering: one clean peak per step.

We then count the steps as the peaks of the filtered signal, and two settings control this. The first is a minimum peak height (min_peak): a peak has to rise above it to count, so small noise bumps are ignored. We set it from the typical height of a real step in our recordings. The second is a minimum time between peaks (min_peak_distance): two peaks closer together than this are treated as one. This stops the small wobbles inside a single step from being counted as extra steps, and it reflects the fact that two real steps cannot be impossibly close together, so we set it to about 0.7 s, from the fastest realistic walking rate. With these two settings a ten-step forward walk gives exactly ten peaks.

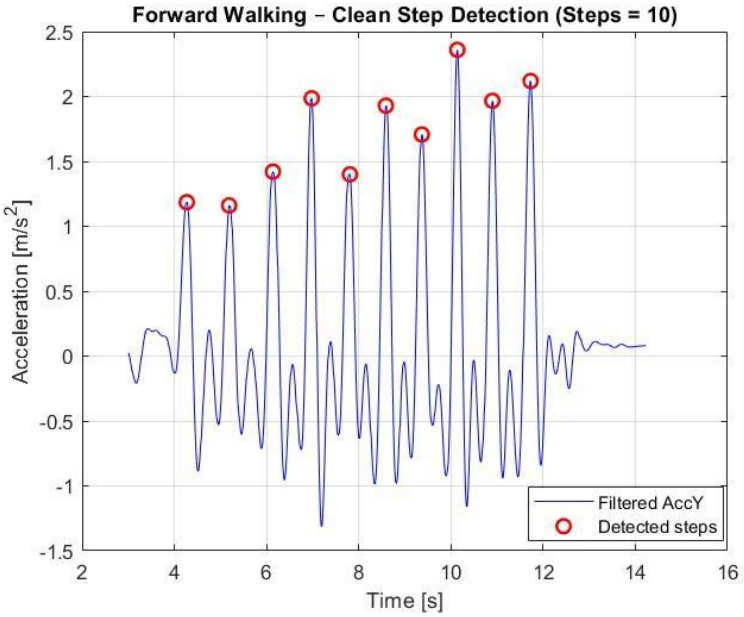
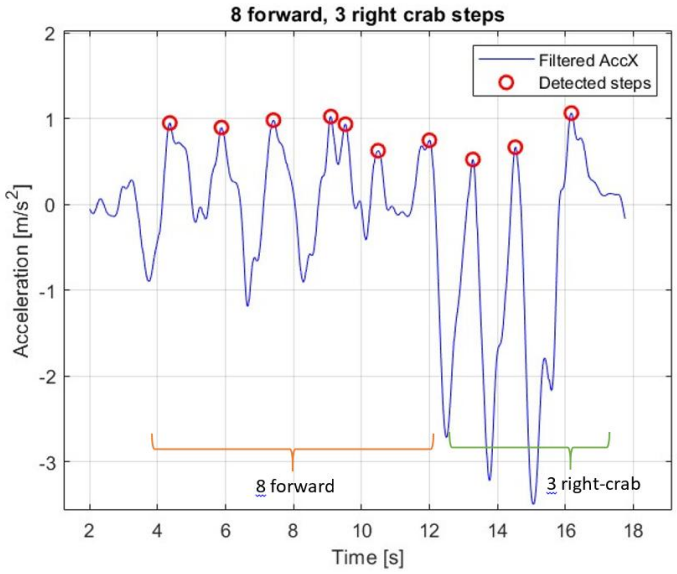


Figure 9. Detected steps (red circles): every peak that passes the height and spacing tests is one step, and all ten are found.

Backward walking gives the same vertical bounce and is counted in the same way. The accelerometer cannot tell backward from forward, This is exactly why we later incorporate the camera.

3.4 Detecting crab steps

For crab steps we look at the lateral (X) axis. We chose this axis because a sideways step pushes the body sideways, and its peak on X is taller than the small sideways sway of normal walking, so we accept it as a crab step only when it is tall enough. In practice forward walking also leaves some motion on the X axis, so the height threshold and the dominance check against the vertical axis are what keep ordinary steps from being read as crab steps.



3.5 Detecting stairs and the elevator

Stairs and the elevator both change the barometer height, but their shapes are different. Stairs give a step-like ramp, while the elevator gives one smooth ramp with no walking steps during the ride. We tell them apart from this difference in shape together with whether steps are happening at the same time.

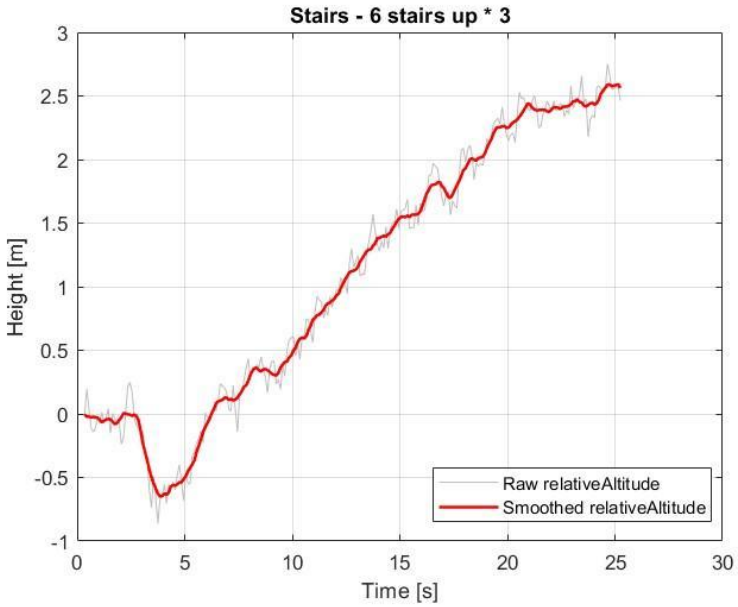


Figure 11. Barometer height while climbing stairs: a step-like rise.

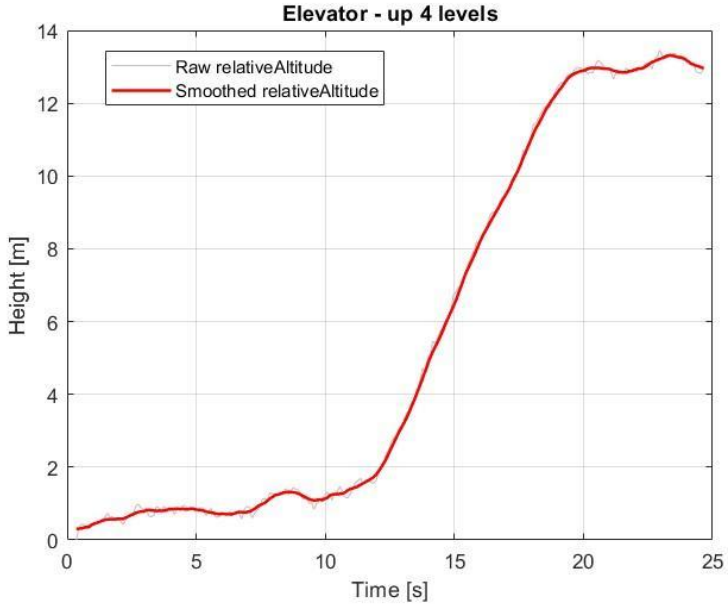


Figure 12. Barometer height during a four-floor elevator ride: one smooth ramp with no steps.

3.6 Detecting turns

A turn is a clear pulse on the gyroscope's yaw axis, so a simple threshold detects it and its sign gives the direction. In the real-time system a turn also takes priority over steps, because the body rotation shakes the accelerometer and could otherwise look like a step.

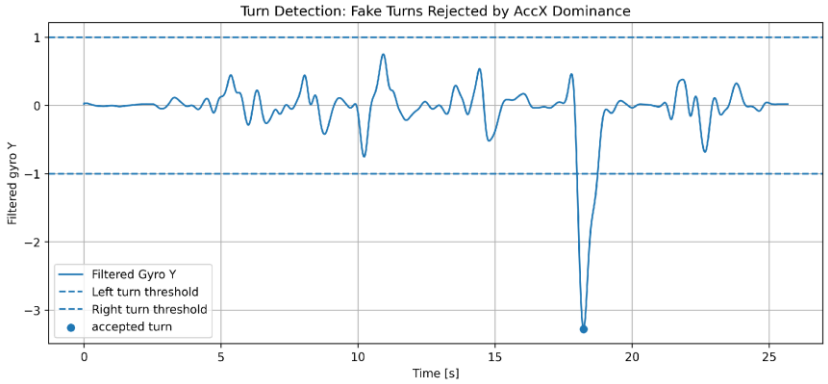


Figure 13. Turn detection on the gyroscope yaw axis: each turn is a clear pulse.

3.7 Reconstructing the 3D path

Finally, feeding all the detected movements into the dead-reckoning update draws the walked path in three dimensions. The offline result below is a mixed route that combines forward walking, a crab segment, a stair descent and an elevator ride, each drawn in its own colour. This offline map is the direct ancestor of the live map in the next chapter.

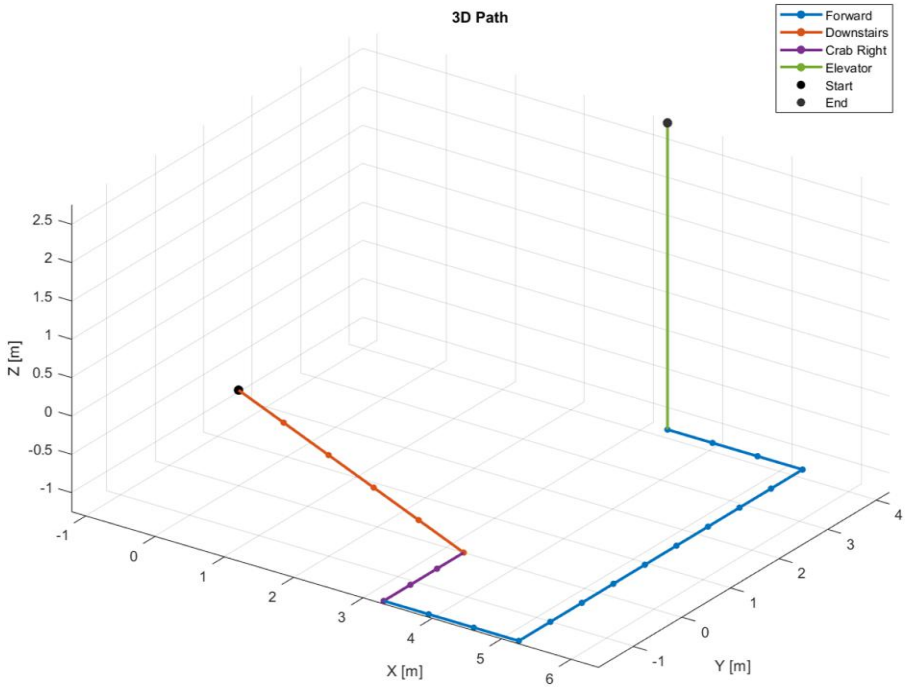


Figure 14. Offline 3D reconstruction of a mixed route (forward, downstairs, crab, elevator).

4. Implementation

4.1 The real-time system

In the second semester we rebuilt everything to run live, in Python, on a laptop we carry during the walk. The phone streams its sensors and camera to the laptop over the connection described in Section 2.7. The laptop polls the phone every 80 ms, keeps the last 10 seconds of data in memory, runs the detection on that window, and confirms each movement only after a short delay, so the same step is never counted twice.

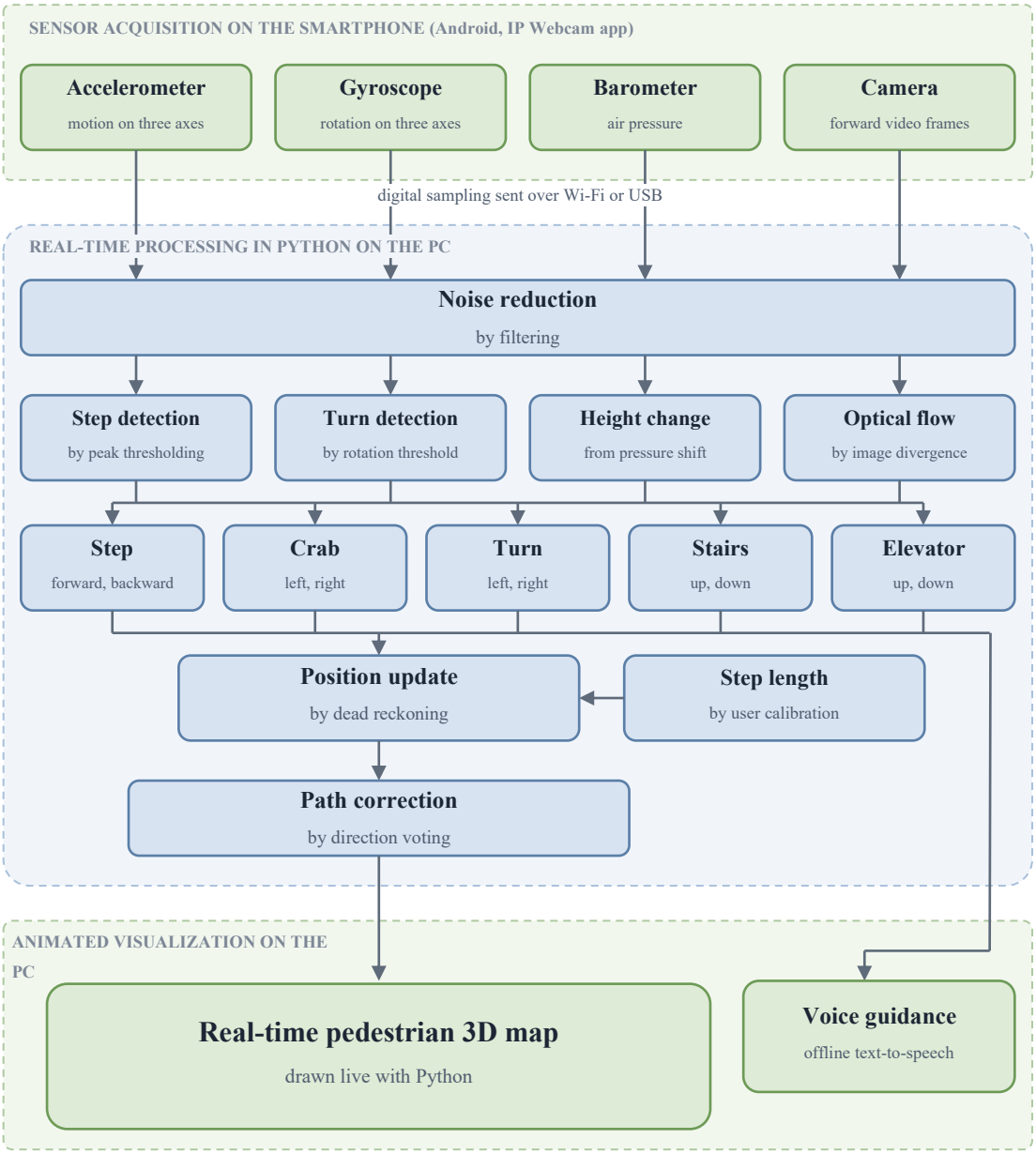


Figure 15. System block diagram

4.2 Detecting steps and turns

Steps and crab steps use the same filter and peak detector we developed offline. We set the step peak height to 0.95 and require a crab step to have a clearly taller lateral peak. Turns take priority over steps: since a turn shakes the accelerometer, we discard step-like peaks that fall right next to a detected turn, which removes a steady source of false steps.

4.3 Deciding direction with the camera

For each step we sum the optical-flow divergence over a short window of frames around the step. In real walks our forward steps measured about +2 to +7 and our backward steps about −7 to −10, with no overlap between them. Forward motion makes the image expand and backward motion makes it contract, as shown below.

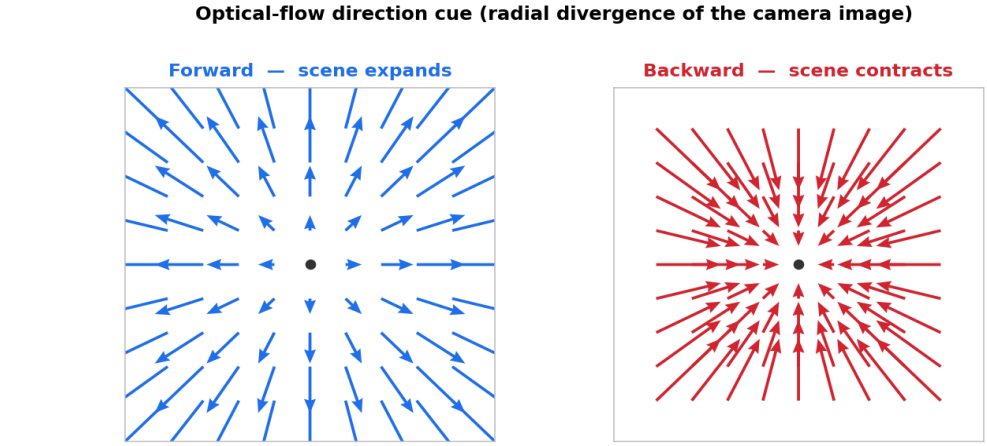


Figure 16. The cue we use for direction: forward motion makes the image expand, and backward motion makes it contract.

We also noticed that the only mistakes happened at the very first or last step of a walk, where starting or stopping rocks the chest. So instead of trusting each step on its own, we decide the direction once per walking segment by a majority vote of its steps, and this fixes those boundary mistakes.

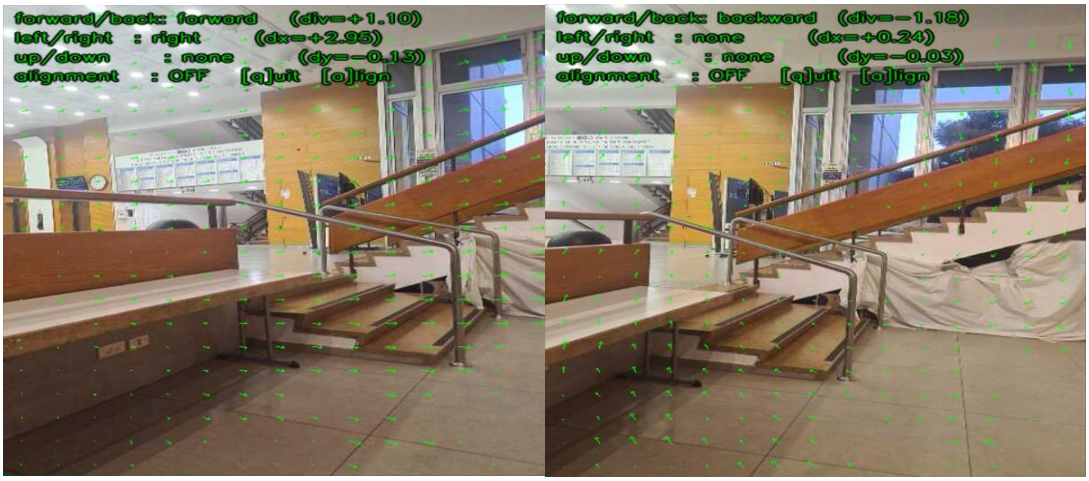


Figure 17. Screenshot of the optical-flow output during a run

4.4 Stairs and the elevator

Because a single stair is the same size as the barometer's noise, we do not try to decide stairs one step at a time. Instead, we look at a whole flight and accept it only if the height rises by more than about 0.55 m. To separate stairs from the elevator we use a simple physical test: a flight of stairs needs roughly one step per stair, while an elevator ride has almost no steps, so a smooth rise with no steps is an elevator. The barometer also reacts about 3 s late, so we let the step window reach back in time to catch the first stairs of a climb.

4.5 Step-length calibration

Because the step length multiplies every step in the map, a wrong value stretches or shrinks the whole path, so we calibrate it once per user. The user walks a known, measured distance, and the program counts the steps with the same detector as the main system and stores the length per step. We found that the longer the calibration walk, the better the estimate. A short walk is easily thrown off by one extra or missing step, while over a longer distance those errors average out. When one of us calibrated over a walk of about 20 m, the result was 0.80 m, which matched the real stride much better than a short calibration did. Our measured step lengths so far are 0.66 m and 0.80 m for the two of us.

4.6 Voice feedback

The system speaks each confirmed movement out loud ("walking", "left turn", "crab right", "up stairs"), which is both a live check for us while testing and the basis for the guidance idea in the motivation. It uses an offline text-to-speech engine (pyttsx3), so it needs no internet and runs entirely on the laptop. The speech runs on its own thread with a short queue, so a slow announcement can never hold up the sensor loop, and if the speech engine is missing the system just keeps running silently.

One choice was deliberate: for a plain step the voice says only "walking", without forward or backward, because the direction can still be corrected by the end-of-run vote, and we did not want the voice to contradict the final map. The stair cue is spoken when the smoothed barometer altitude has risen by more than a flight over the last few seconds while steps are happening (a rise with no steps is an elevator, and stays silent).

4.7 Outputs

Every run saves the path as a CSV file, the raw sensor logs so we can replay any run offline, and a set of figures: the filtered signals with the detected movements marked, and the 3D and 2D maps of the walked path.

4.8 Main parameters

The main tuned values of the system and how we set each one:

Parameter	Value	How we set it
Step peak height / spacing	0.95 / 0.7 s	tuned on our own walks
Crab peak height / dominance	1.78 / ×1.20	tuned on our own walks
Turn threshold	1.0 rad/s	tuned on our own walks
Poll interval / window	80 ms / 10 s	chosen for real-time response
Camera fetch rate	≈4 / s	faster fetching slowed the sensor polling

Stair riser height	0.17 m	measured (2.20 m over 13 stairs)
Flight minimum rise	0.55 m	above the barometer's noise
Elevator step fraction	0.30	a ride has almost no steps (0.02 vs ~1.0)
Default step length	0.70 m	fallback when a user is not calibrated

5. Results and tests

We measured the system against the three error metrics defined for the project:

$$E_{steps} = \frac{|N_{detected} - N_{true}|}{N_{true}} * 100\%$$

$$E_{class} = \frac{wrong\ movements}{total\ movements} * 100\%$$

$$E_{path} = \frac{|position\ system - position\ true|}{route\ length} * 100\%$$

5.1 Step count and distance

On a measured 20.5 m straight walk the system detected 25 steps for 26 real steps, a 0.96% step-count error. With the calibrated step length of 0.80 m this gives an estimated distance of 20 m against the real 20.5 m, a 0.5 m error over 20.5 m, which is well inside the target of 1 m per 10 m.

Quantity	True	System	Error
Steps	26	25	0.96%
Distance	20.5 m	20 m	0.5 m

5.2 Movement classification

We walked a mixed route of 30 movements (forward, backward, a turn, crab steps, an elevator up and a stair descent). The system classified 29 of them correctly, which is 96.7%. The single error was one forward step at the top of the stair descent that got counted as part of the flight. The live 3D map of this route is shown below.

Movement	Walked	Detected	Correct
Forward	11	10	10
Backward	3	3	3
Left turn	1	1	1
Left crab	3	3	3
Elevator up	1	1	1
Stairs down	11	12	11
Total	30	30	29 (96.7%)

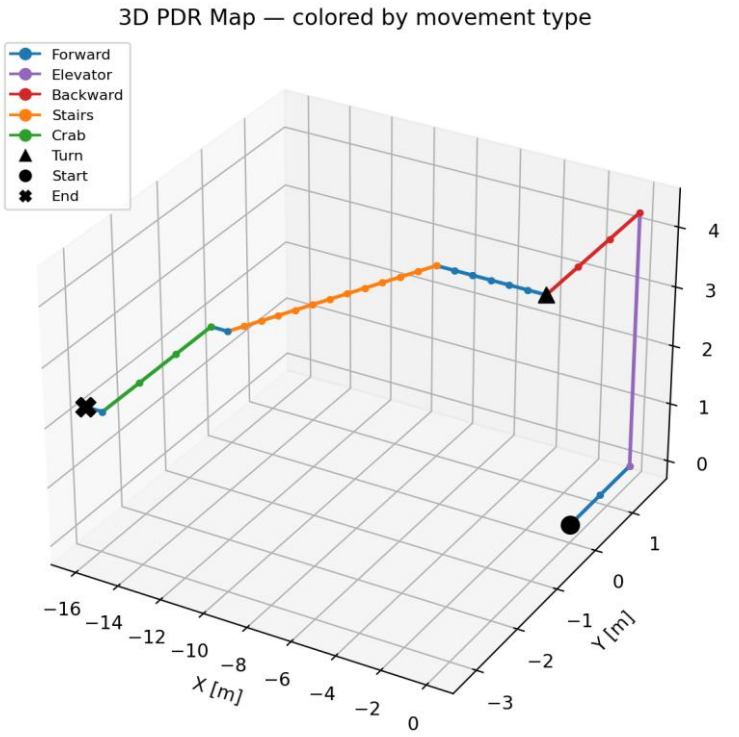


Figure 18. Live 3D map of the mixed route (96.7% classification).

5.3 Stairs and the elevator

A dedicated stair run detected all 18 stairs and every other movement correctly (100% classification), with a measured climb of +3.06 m. In a separate test the four-floor elevator ride was correctly identified as an elevator, not stairs, with a rise of +11.29 m. This was the test that was impossible over Wi-Fi and only became possible once we switched to the cable.

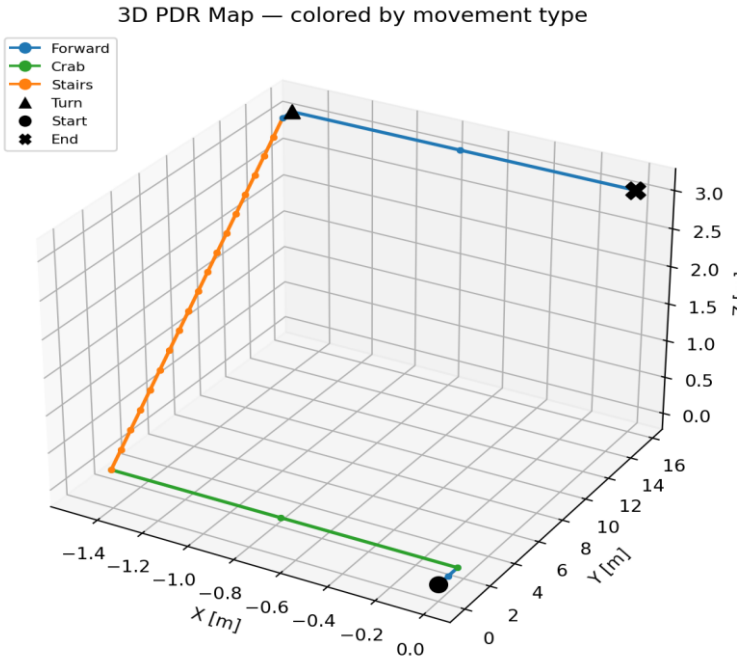


Figure 19. Live 3D map of the stair run: flat walking, a crab segment, then 18 stairs up.

5.4 Status against the objectives

Objective	Target	Result	Status
Step-count error	≤ 5%	1.2%	met
Classification	≥ 85%	96.9%	met
Path error	≤ 1 m / 10 m	0.24m per 10m	met
Users	≥ 4	4 calibrated	met

The results were calculated from the test data presented in the table below:

#	Run	Real	Detected	Path error	Steps (det / walked)	Classification
1	Straight, 20.5 m	20.5 m	20.0 m	0.50 m (2.4%)	25 / 26 (3.8%)	25 / 25 (100%)
2	Straight, 28.5 m	28.5 m	28.8 m	0.30 m (1.1%)	36 / 36 (0%)	36 / 36 (100%)
3	Straight, 28.5 m, take 2	28.5 m	27.2 m	1.30 m (4.6%)	34 / 35 (2.9%)	34 / 34 (100%)
4	Square, counter-clockwise	19.6 m	19.2 m	0.40 m (2.0%)	24 / 24 (0%)	28 / 28 (100%)
5	Square, clockwise	19.7 m	19.2 m	0.50 m (2.5%)	24 / 24 (0%)	27 / 27 (100%)
6	Square, clockwise, 2 laps	32.6 m	32.0 m	0.60 m (1.8%)	40 / 40 (0%)	43 / 48 (89.6%)
7	Mixed route 1	n/a	n/a	n/a	n/a	29 / 30 (96.7%)
8	Mixed route 2	n/a	n/a	n/a	n/a	25 / 26 (96.2%)
9	Mixed route 3	n/a	n/a	n/a	n/a	30 / 32 (93.8%)

6. Summary, conclusions and further work

6.1 Summary

We built a working real-time system that tracks a pedestrian indoors without GPS. A chest-worn phone streams its sensors and camera to a laptop, which classifies steps, crab steps, turns, stairs and elevator rides and draws the path on a live 3D map, with per-user step length and voice feedback. On our documented runs the system met the step-count, classification and distance targets.

6.2 What we learned

The biggest lesson was that the sensors decide the algorithm, not the other way around. Measuring the barometer's noise, which is about one stair high, is what told us to detect whole flights instead of single stairs, and watching the camera fail only at the start and end of a walk is what led us to decide direction once per segment. The second lesson was very practical. The connection we started with, Wi-Fi, was the least reliable part of the whole system, and switching to a cable turned it into the most reliable part and was the only way we could measure the elevator at all.

6.3 Further work

- Turns at any angle, by integrating the gyroscope (and possibly using the magnetometer).
- Removing the fixed chest-mount assumption with phone-orientation tracking.
- Running everything on the phone, without the laptop and cable.
- Snapping the path to a building floor plan to bound the drift.
- A full guidance mode for visually impaired users, building on the voice output.

7. Project documentation

The full source code of the system is available on GitHub, at:
https://github.com/rabeeabul/Pedestrian-Navigation-without-GPS
The repository includes a README file with detailed instructions for installing the required packages, connecting the phone, calibrating the step length and running the program.

References

- Android Developers, Sensors overview, https://developer.android.com/develop/sensors-and-location/sensors/sensors_overview
- Optical Flow, Wikipedia, https://en.wikipedia.org/wiki/Optical_flow
- P. Khlebovich, "IP Webcam" (Android application), https://play.google.com/store/apps/details?id=com.pas.webcam
- K. Choi, "Sensor Logger" (Android application), https://www.tszheichoi.com/sensorlogger
- SciPy documentation, signal processing (butter, filtfilt, find_peaks), https://docs.scipy.org/doc/scipy/reference/signal.html
- GeeksforGeeks, Text-to-Speech in Python using pyttsx3, https://www.geeksforgeeks.org/python/convert-text-speech-python/